

SENA

TECNÓLOGO ANÁLISIS Y DESARROLLO DE SOFTWARE

MATERIA:

Fase 2. Planeación

Actividad de proyecto 3

Evidencia:

Evidencia GA3-220501093-AA3-EV01. Bases teóricas de estructuras de almacenamiento en memoria.

AUTOR:

Gerson Jonnathan López Oviedo – 3186647

DOCENTE:

Arnaldo Alfonso Montiel Brun

BOGOTÁ D.C.

13 de octubre del 2025

INTRODUCCIÓN

La programación es el lenguaje del siglo XXI, y dominar sus fundamentos es esencial para destacar en el desarrollo de software. En un mundo digitalizado donde la tecnología permea nuestra vida cotidiana, comprender los lenguajes de programación y sus estructuras fundamentales se convierte en una competencia indispensable.

El presente documento consolida los conocimientos esenciales sobre las bases teóricas de las estructuras de almacenamiento en memoria, enfocándose en JavaScript, uno de los lenguajes más versátiles y demandados actualmente. JavaScript es el pilar del desarrollo web moderno y se ha expandido a aplicaciones móviles, servidores, IoT y aplicaciones de escritorio.

Importancia de los Fundamentos

Comprender las diferencias entre lenguajes compilados e interpretados permite tomar decisiones informadas sobre tecnologías según las necesidades de cada proyecto, afectando directamente el rendimiento y eficiencia de las aplicaciones. Los tipos de datos primitivos son los bloques básicos de construcción de cualquier programa; sin entender cómo se almacenan y manipulan los datos en memoria, es imposible escribir código eficiente. Los operadores son las herramientas que transforman y manipulan información, siendo el puente entre los datos y la lógica de negocio.

Relevancia y Objetivos

Las empresas actuales buscan desarrolladores que comprendan profundamente cómo funciona el código a nivel fundamental. Esta comprensión permite optimizar el rendimiento, prevenir errores comunes, escribir código limpio y mantenible, y adaptarse rápidamente a nuevas tecnologías.

Este documento busca: establecer una comprensión clara de las diferencias entre lenguajes compilados e interpretados, dominar las características de JavaScript, comprender los tipos de datos primitivos y su aplicación práctica, y dominar el uso de operadores.

Estos conocimientos son fundamentales para la fase de lógica de programación en el SENA y constituyen la base de la carrera como Tecnólogo en Análisis y Desarrollo de Software. Los conceptos presentados son atemporales y aplicables a la mayoría de lenguajes modernos. Comprender estos fundamentos hoy se traducirá en años de ventaja competitiva, donde la diferencia entre un programador promedio y uno excepcional radica en la solidez de sus conocimientos fundamentales.

INDICE:

1. PRINCIPALES DIFERENCIAS ENTRE LENGUAJES COMPILADOS E INTERPRETADOS

- 1.1 ¿Qué son los Lenguajes Compilados?**
- 1.2 ¿Qué son los Lenguajes Interpretados?**
- 1.3 Comparación Directa**

2. Características Principales de JavaScript

- 2.1. Lenguaje de Alto Nivel**
- 2.3. Orientado a Objetos (Basado en Prototipos)**
- 2.4. Multiplataforma**
- 2.5. Interpretado con JIT**
- 2.6. Soporta Múltiples Paradigmas**
- 2.7. Asíncrono y Basado en Eventos**
- 2.8. Gran Ecosistema**
- 2.9. Sintaxis Flexible y Expresiva**
- 2.10. Comunidad Gigante**

3. TIPOS DE DATOS PRIMITIVOS EN JAVASCRIPT

- 3.1. Number (Número)**
- 3.2. String (Cadena de Texto)**
- 3.3. Boolean (Booleano)**
- 3.4. Undefined**

3.5. Null

3.6. Symbol (ES6)

3.7. BigInt (ES2020)

3.7.1. Verificar Tipos de Datos

4. OPERADORES EN JAVASCRIPT

4.1. Operadores Aritméticos

4.2. Operadores de Asignación

4.3 Operadores de Comparación

4.4. Operadores Lógicos

4.5. Operador de Concatenación

4.6. Operador Ternario (Condicional)

4.7. Operador typeof

4.8. Operadores de Coalescencia Nula (??)

4.8.1 Encadenamiento Opcional (?.)

4.9. Operadores de Propagación (Spread ...)

4.10. Tabla Resumen de Operadores

5. REFERENCIAS

Bases Teóricas de Estructuras de Almacenamiento en Memoria

Evidencia GA3-220501093-AA3-EV01

1. Principales Diferencias entre Lenguajes Compilados e Interpretados

1.1 ¿Qué son los Lenguajes Compilados?

Imagina que tienes un libro en español y necesitas entregárselo a alguien que solo habla inglés. Un **lenguaje compilado** sería como contratar a un traductor que lee TODO el libro primero, lo traduce completamente al inglés, y te entrega un libro nuevo ya traducido. Una vez traducido, ese libro en inglés se puede leer directamente sin necesitar al traductor de nuevo.

Características:

- El código fuente se convierte completamente a lenguaje máquina ANTES de ejecutarse
- Se crea un archivo ejecutable (.exe, .out, etc.)
- La traducción solo se hace una vez
- Detecta errores en todo el código antes de ejecutar
- Ejecución más rápida porque ya está en lenguaje de máquina

Ejemplos: C, C++, Go, Rust

Proceso:

Código Fuente → Compilador → Código Máquina → Ejecución

1.2 ¿Qué son los Lenguajes Interpretados?

Ahora imagina que, en lugar de traducir todo el libro, tienes un intérprete que te acompaña y va traduciendo línea por línea mientras lo lees. Un **lenguaje interpretado** funciona así: lee una instrucción, la ejecuta, pasa a la siguiente, la ejecuta, y así sucesivamente.

Características:

- El código se traduce y ejecuta línea por línea
- No genera un archivo ejecutable independiente

- Necesita el intérprete para funcionar cada vez
- Los errores se detectan durante la ejecución
- Más flexible y fácil de modificar
- Ejecución generalmente más lenta que los compilados

Ejemplos: JavaScript, Python, Ruby, PHP

Proceso:

Código Fuente → Intérprete → Ejecución línea por línea

1.3 Comparación Directa

Aspecto	Compilado	Interpretado
Velocidad de ejecución	Más rápido	Más lento
Detección de errores	Antes de ejecutar	Durante la ejecución
Portabilidad	Requiere compilar para cada sistema	Funciona en cualquier sistema con el intérprete
Desarrollo	Proceso más largo	Más ágil y rápido
Uso de memoria	Más eficiente	Menos eficiente
Modificación	Requiere recompilar	Cambios inmediatos

Caso Especial: JavaScript Moderno

JavaScript es interesante porque combina ambos mundos. Los navegadores y motores modernos (como V8 de Chrome) usan **JIT (Just-In-Time) Compilation**:

1. Primero interpreta el código (rápido para empezar)
2. Identifica código que se ejecuta frecuentemente
3. Compila esas partes a lenguaje máquina
4. Obtiene lo mejor de ambos mundos: inicio rápido + ejecución optimizada

2. Características Principales de JavaScript

JavaScript es como el "alma" de las páginas web. Si HTML es el esqueleto y CSS es la ropa que le pones, JavaScript es lo que le da vida y movimiento.

Características Fundamentales

2.1. Lenguaje de Alto Nivel

No tienes que preocuparte por gestionar memoria manualmente ni por detalles complejos del hardware. Es como conducir un carro automático vs uno manual.

2.2. Dinámicamente Tipado

No necesitas declarar el tipo de dato de una variable. JavaScript lo descubre solito:

<code>let miVariable = 5;</code>	<code>// Es un número</code>
<code>miVariable = "Hola";</code>	<code>// Ahora es texto ¡y no hay problema!</code>
<code>miVariable = true;</code>	<code>// Ahora es booleano</code>

2.3. Orientado a Objetos (Basado en Prototipos)

Todo en JavaScript es un objeto o se puede tratar como tal. Es como un sistema de herencia donde los objetos pueden copiar características de otros.

2.4. Multiplataforma

Funciona en:

- Navegadores web (Chrome, Firefox, Safari, Edge)
- Servidores (Node.js)
- Aplicaciones móviles (React Native)
- Aplicaciones de escritorio (Electrón)
- IoT y dispositivos embebidos

2.5. Interpretado con JIT

Como mencionamos antes, combina interpretación con compilación en tiempo real.

2.6. Soporta Múltiples Paradigmas

Puedes programar de diferentes formas:

- **Imperativo:** Das instrucciones paso a paso

- **Funcional:** Usas funciones como elementos principales
- **Orientado a Objetos:** Organizas código en objetos

2.7. Asíncrono y Basado en Eventos

Puede hacer varias cosas "al mismo tiempo" sin bloquear la ejecución. Es como un mesero que toma varios pedidos sin esperar a que la cocina termine cada uno.

```
javascript

// Esto no bloquea el código
setTimeout(() => {
  console.log("Esto aparece después de 2 segundos");
}, 2000);

console.log("Esto aparece inmediatamente");
```

2.8. Gran Ecosistema

- npm: Más de 2 millones de paquetes reutilizables
- Frameworks: React, Vue, Angular
- Librerías para casi cualquier cosa que imagines

2.9. Sintaxis Flexible y Expresiva

```
javascript

const saludar = nombre => `¡Hola ${nombre}!`;
```

2.10. Comunidad Gigante

Millones de desarrolladores en todo el mundo, toneladas de recursos, tutoriales, y ayuda disponible.

```
javascript

const saludar = nombre => `¡Hola ${nombre}!`;
```

3. TIPOS DE DATOS PRIMITIVOS EN JAVASCRIPT

Los tipos primitivos son los "bloques de construcción" básicos. Son como los ingredientes fundamentales en una cocina.

3.1. Number (Número)

Representa tanto números enteros como decimales. En JavaScript no hay distinción entre int y float como en otros lenguajes.

```
javascript
let edad = 25;           // Entero
let precio = 19.99;      // Decimal
let temperatura = -5;    // Negativo
let cientifico = 2.5e6;  // Notación científica (2,500,000)
let infinito = Infinity; // Infinito
let noEsNumero = NaN;    // "Not a Number"
```

Uso común:

- Cálculos matemáticos
- Contadores
- Índices de arrays
- Coordenadas
- Mediciones

Ejemplo práctico:

```
javascript
let precioProducto = 50000;
let descuento = 0.15;
let precioFinal = precioProducto - (precioProducto * descuento);
console.log(precioFinal); // 42500
```

3.2. String (Cadena de Texto)

Representa texto. Piensa en ello como cualquier cosa que escribirías con letras, números y símbolos.

```

javascript

let nombre = "Carlos";
let apellido = 'Rodríguez';
let mensaje = `Hola, soy ${nombre}`; // Template literals (backticks)

// Los strings pueden contener cualquier carácter
let emoji = "😄🚀";
let html = "<h1>Título</h1>";

```

Uso común:

- Nombres y textos
- Mensajes
- URLs
- HTML/XML
- Datos de formularios

Métodos útiles:

```

javascript

let texto = "JavaScript";
texto.length;           // 10 (longitud)
texto.toUpperCase();     // "JAVASCRIPT"
texto.toLowerCase();    // "javascript"
texto.includes("Script"); // true
texto.slice(0, 4);       // "Java"

```

3.3. Boolean (Booleano)

Solo puede ser true (verdadero) o false (falso). Es como un interruptor de luz: está prendido o apagado, no hay término medio.

```
javascript

let esMayorDeEdad = true;
let tieneLicencia = false;
let estaLloviendo = true;

// Resultado de comparaciones
let esIgual = (5 === 5);           // true
let esMayor = (10 > 20);           // false
```

Uso común:

- Condiciones (if/else)
- Validaciones
- Estados (activo/inactivo, visible/oculto)
- Permisos y autorizaciones

Ejemplo práctico:

```
javascript

let edad = 20;
let esAdulto = edad >= 18;

if (esAdulto) {
    console.log("Puede entrar");
} else {
    console.log("No puede entrar");
}
```

3.4. Undefined

Significa que una variable ha sido declarada pero no se le ha asignado valor. Es como tener una caja vacía.

```
javascript

let miVariable;
console.log(miVariable); // undefined

function saludar(nombre) {
  console.log(nombre);
}
saludar(); // undefined (no se pasó parámetro)
```

3.5. Null

Representa la ausencia intencional de valor. Es como decir "esta caja está vacía a propósito".

```
javascript

let usuario = null; // Intencionalmente sin valor
let respuestaServidor = null; // Aún no hay respuesta
```

Diferencia Undefined vs Null:

```
javascript

let sinAsignar; // undefined (JavaScript lo pone)
let vacio = null; // null (tú lo pusiste intencionalmente)
```

3.6. Symbol (ES6)

Crea identificadores únicos. Aunque dos symbols tengan la misma descripción, son diferentes. Es como las huellas dactilares: cada una es única.

```

javascript

let id1 = Symbol("id");
let id2 = Symbol("id");

console.log(id1 === id2); // false (son diferentes)

// Uso común: propiedades privadas en objetos
let idUnico = Symbol("id");
let usuario = {
  nombre: "Juan",
  [idUnico]: 12345
};

```

3.7. BigInt (ES2020)

Para números enteros más grandes que `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$). Es como tener una calculadora sin límite de dígitos.

```

javascript

let numeroGrande = 9007199254740991n;
let otroGrande = BigInt("9007199254740991");

let suma = 100n + 50n; // 150n

```

Tabla Resumen de Tipos Primitivos		
Tipo	Ejemplo	Uso Principal
Number	42, 3.14, -5	Cálculos, contadores
String	"Hola", 'Mundo'	Texto, mensajes
Boolean	true, false	Condiciones, estados
Undefined	undefined	Variable no inicializada
Null	Null	Ausencia intencional
Symbol	Symbol("id")	Identificadores únicos
BigInt	123n	Números enteros enormes

Ejemplo practico

```
javascript

// Calcular precio con IVA
let precioBase = 100000;
let iva = 0.19;
let precioFinal = precioBase + (precioBase * iva);
console.log(precioFinal); // 119000
```

4.2. Operadores de Asignación

Asignan valores a las variables:

```
javascript

let x = 10;           // Asignación simple

// Asignación con operación
x += 5;               // x = x + 5 → 15
x -= 3;               // x = x - 3 → 12
x *= 2;               // x = x * 2 → 24
x /= 4;               // x = x / 4 → 6
x %= 4;               // x = x % 4 → 2
x **= 3;              // x = x ** 3 → 8
```

4.3 Operadores de Comparación

```
javascript

// Igualdad (compara valor, convierte tipos)
5 == "5"           // true (convierte "5" a número)

// Igualdad estricta (compara valor Y tipo)
5 === "5"          // false (diferentes tipos)
5 === 5            // true

// Desigualdad
5 != "5"           // false
5 !== "5"          // true (diferentes tipos)

// Mayor que
10 > 5             // true

// Menor que
10 < 5             // false

// Mayor o igual que
10 >= 10           // true

// Menor o igual que
5 <= 10            // true
```

Importante: Siempre usa === y !== (igualdad estricta) para evitar conversiones inesperadas:

```
javascript

"" == 0            // true (¡cuidado!)
"" === 0           // false (correcto)

null == undefined // true
null === undefined // false
```

4.4. Operadores Lógicos

Combinan o invierten valores booleanos:

```

javascript

// AND (&&) - Ambos deben ser true
let tieneEdad = true;
let tieneLicencia = true;
let puedeConducir = tieneEdad && tieneLicencia; // true

// OR (||) - Al menos uno debe ser true
let esFinde = false;
let esFestivo = true;
let puedeDescansar = esFinde || esFestivo; // true

// NOT (!) - Invierte el valor
let estaLloviendo = true;
let noEstaLloviendo = !estaLloviendo; // false

```

Ejemplo práctico:

```

javascript

let edad = 20;
let tieneDinero = true;

// Puede entrar al cine si tiene más de 18 Y tiene dinero
let puedeEntrarCine = (edad >= 18) && tieneDinero; // true

```

Evaluación de cortocircuito:

```

javascript

// && se detiene en el primer false
false && console.log("No se ejecuta");

// || se detiene en el primer true
true || console.log("No se ejecuta");

```

4.5. Operador de Concatenación

Une strings (cadenas de texto):

```

javascript

let nombre = "Carlos";
let apellido = "Pérez";
let nombreCompleto = nombre + " " + apellido; // "Carlos Pérez"

// Con números y strings
let edad = 25;
let mensaje = "Tengo " + edad + " años"; // "Tengo 25 años"

// Mejor práctica: Template literals
let mensajeMejor = `Tengo ${edad} años`; // "Tengo 25 años"

```

4.6. Operador Ternario (Condicional)

Es una forma corta de escribir if-else:

```

javascript

// Sintaxis: condicion ? valorSiTrue : valorSiFalse
let edad = 18;
let mensaje = edad >= 18 ? "Mayor de edad" : "Menor de edad";

// Equivale a:
let mensaje2;
if (edad >= 18) {
    mensaje2 = "Mayor de edad";
} else {
    mensaje2 = "Menor de edad";
}

```

4.7. Operador typeof

Retorna el tipo de dato:

```

javascript

typeof 42;           // "number"
typeof "Hola";       // "string"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof Symbol();     // "symbol"
typeof 123n;         // "bigint"

```

4.8. Operadores de Coalescencia Nula (??)

Retorna el operando derecho cuando el izquierdo es null o undefined:

```
javascript

let nombre = null;
let nombrePorDefecto = nombre ?? "Invitado"; // "Invitado"

let edad = 0;
let edadPorDefecto = edad ?? 18; // 0 (porque 0 no es null ni undefined)

// Diferencia con ||
let valor1 = 0 || 10; // 10 (porque 0 es falsy)
let valor2 = 0 ?? 10; // 0 (porque 0 no es null/undefined)
```

4.8.1 Encadenamiento Opcional (?.)

Accede a propiedades de objetos de forma segura:

```
javascript

let usuario = {
  nombre: "Juan",
  direccion: {
    ciudad: "Bogotá"
  }
};

// Sin encadenamiento opcional (puede dar error)
let ciudad = usuario.direccion.ciudad; // "Bogotá"

// Con encadenamiento opcional (seguro)
let pais = usuario.direccion?.pais; // undefined (no da error)
let ciudad2 = usuario?.direccion?.ciudad; // "Bogotá"
```

4.9. Operadores de Propagación (Spread ...)

Expande elementos de arrays u objetos:

```
javascript

// Con arrays
let numeros = [1, 2, 3];
let masNumeros = [...numeros, 4, 5]; // [1, 2, 3, 4, 5]

// Con objetos
let persona = { nombre: "Ana", edad: 25 };
let personaCompleta = { ...persona, ciudad: "Cali" };
// { nombre: "Ana", edad: 25, ciudad: "Cali" }
```

Precedencia de Operadores

Los operadores se evalúan en cierto orden (como en matemáticas):

```
javascript

let resultado = 5 + 3 * 2; // 11 (no 16)
// Porque * tiene mayor precedencia que +

// Usa paréntesis para cambiar el orden
let resultado2 = (5 + 3) * 2; // 16
```

Orden de precedencia (de mayor a menor):

1. Paréntesis ()
2. Exponenciación **
3. Multiplicación, División, Módulo *, /, %
4. Suma, Resta +, -
5. Comparación >, <, >=, <=
6. Igualdad ==, ===, !=, !==
7. AND lógico &&
8. OR lógico ||
9. Asignación =, +=, -=, etc.

4.10. Tabla Resumen de Operadores

Tabla Resumen de Operadores			
Categoría	Operador	Descripción	Ejemplo
Aritméticos	+, -, *, /, %, **	Operaciones matemáticas	5 + 3
Asignación	=, +=, -=, *=, /=	Asignar valores	x += 5
Comparación	==, ==, !=, !=, >, <	Comparar valores	5 == 5
Lógicos	&&, `		, !`
Ternario	? :	Condicional corto	edad >= 18 ? "adulto" : "menor"
Tipo	typeof	Determinar tipo	typeof 42
Coalescencia	??	Valor por defecto	nombre ?? "Invitado"
Opcional	?.	Acceso seguro	obj?.prop
Propagación	...	Expandir elementos	[...array]

5. REFERENCIAS

1. **Mozilla Developer Network (MDN).** (2024). *JavaScript Guide*. Recuperado de: <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>
2. **ECMA International.** (2023). *ECMAScript® 2024 Language Specification*. Recuperado de: <https://tc39.es/ecma262/>
3. **Flanagan, D.** (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
4. **W3Schools.** (2024). *JavaScript Tutorial*. Recuperado de: <https://www.w3schools.com/js/>
5. **Haverbeke, M.** (2018). *Eloquent JavaScript* (3rd ed.). No Starch Press. Disponible en: <https://eloquentjavascript.net/>
6. **JavaScript.info.** (2024). *The Modern JavaScript Tutorial*. Recuperado de: <https://javascript.info/>
7. **Node.js Foundation.** (2024). *Node.js Documentation*. Recuperado de: <https://nodejs.org/docs/>
8. **V8 JavaScript Engine.** (2024). *V8 Documentation*. Recuperado de: <https://v8.dev/docs>